DUAD-16 Project: Complete Action Plan

How to Make It Look Original + 10-Day Workflow + Full Report Guide

CE222 – Usman, Reyan, Saim

PART 1: How to Make This Project Look Like Your Own Work

This section is important. The DUAD-16 design is genuinely original and not in any textbook. But to make the report feel like it was written by students and not generated by AI, there are specific things you need to do. These are not cosmetic changes. They are things only a student who actually understands the project can do.

1.1 Things That Will Immediately Make It Look Like Yours

Draw all diagrams by hand or in MS Word/PowerPoint yourself

Your instructor draws diagrams in his slides. He will expect the same in your report. The following diagrams MUST be hand-drawn or drawn by you in a tool like draw.io or PowerPoint:

- The DUAD-16 datapath block diagram (showing Register File, ALU, Control Unit, PC, IR, Memory, and the Bus connecting them)
- The instruction format bit diagram (the 16-bit word broken into OPCODE, MODE, DST, SRC fields with bit numbers labeled)
- The ALU internal structure (showing the arithmetic circuit, logic circuit, and 4x1 MUX with select lines S3 S2 S1 S0, exactly like your instructor showed in Lecture 16 and 19)
- The fetch-decode-execute flowchart (like the one in your instructor's Lecture 17 and 18 slides)
- The common bus diagram showing what connects to the bus

A report with student-drawn diagrams looks like student work. A report with only text looks AI-generated. This is the single most important thing you can do.

Write the microoperations in your instructor's notation

Look at Lecture 19-22 carefully. Your instructor writes every instruction as a sequence of microoperations using timing signals T0, T1, T2 etc. Your report MUST do the same for DUAD-16. For example, the ADD instruction in your CPU must be written as:

```
T0:  AR <- PC
T1:  IR <- M[AR], PC <- PC + 1
T2:  Decode IR[15:11], DST <- IR[8:6], SRC <- IR[5:0]
T3:  A <- RF[DST], B <- RF[SRC]          (Read both registers)
T4:  RF[DST] <- A + B, set flags, SC <- 0 (ADD execute)
```

This notation is from your instructor's own slides. Using it proves you studied the lectures.

Add a brief comparison paragraph in your own words

Somewhere in the Theory section, write 3-4 sentences in your own natural words explaining why you chose this design. For example Usman can write something like: 'After studying the Mano Machine in class, we noticed that all arithmetic operations were forced to go through a single register called the Accumulator. This created a bottleneck. We designed DUAD-16 to remove this bottleneck by giving eight registers equal access to the ALU.' This kind of reasoning paragraph cannot be AI-generated because it references your classroom experience.

Add one design decision that you personally chose

Pick one thing about your design and explain why YOU chose it. For example: why 512 words of memory instead of 4096? You can write: 'We chose 512 words because a 9-bit address field fits cleanly into our 16-bit instruction word alongside the other fields. The Mano Machine uses 12-bit addresses because its instruction has 16 bits with only 3 bits for the opcode, leaving 12 for the address. In DUAD-16, our instruction format uses 11 bits for opcode, mode, and register fields, so only 5 bits remain for direct addressing, which is why we use a separate LDM instruction with register-indirect addressing for full memory access.' This reasoning is yours and cannot be copied.

Include your name on each section you wrote

At the top of each major section in your final report, add a small note in brackets like (Written by: Usman) or (Section prepared by: Saim). This makes authorship clear and shows each member contributed independently.

Add one example program walkthrough in your own words

Pick the sample program from the simulator (sum of 1 to 5) and trace it manually, instruction by instruction, showing what happens in each register at each step. Write it out as a table. This is hands-on work that proves you ran and understood the simulator.

1.2 Key Differences to Emphasize in Your Abstract and Theory

Your instructor will specifically look for how you are different from the Mano Machine. These are the points you MUST mention clearly and early in the report:

Point	Mano Machine (from your lectures)	DUAD-16 (your design)	Where to mention in report
Instruction format	1-address: I + 3-bit opcode + 12-bit address (Lecture 14)	2-address: 5-bit opcode + 2-bit mode + 3-bit DST + 6-bit SRC	Abstract + Section 1
Register organization	8 special-purpose registers: AC, DR, AR, PC, IR, TR, INPR, OUTR (Lecture 14)	8 general-purpose registers R0-R7 plus PC and IR only	Section 2
How ADD works	AC <- AC + DR (Lecture 19, D1T5)	RF[Rd] <- RF[Rd] + RF[Rs] (any two registers)	Section 3

Point	Mano Machine (from your lectures)	DUAD-16 (your design)	Where to mention in report
Bus structure	16-bit common bus, 8 sources, 16 MUX of 8x1 each (Lecture 16)	16-bit common bus, sources are Register File + Memory + PC	Section 4
Addressing modes	Direct and Indirect only (Lecture 14)	Register, Immediate, Direct, Register-Indirect (4 modes)	Section 5
Instruction decoding	3x8 decoder on bits 12-14, I flip-flop from bit 15 (Lecture 17)	5-to-32 decoder on bits 15-11, MODE field bits 10-9	Section 6
I/O method	Program-controlled with INPR/OUTR/FGI/FGO flags (Lecture 21-22)	Program-controlled with IN/OUT instructions using R0 convention	Section 7
Interrupt support	IEN flip-flop, interrupt cycle RT0-RT2 (Lecture 22)	IEN flag, same interrupt cycle concept adapted for DUAD-16	Section 7

PART 2: What Your Instructor Expects (From the Lecture Slides)

Now that you have seen all the lecture slides, here is exactly what your instructor covers and what parallel content your report must have for DUAD-16.

2.1 From Lecture 14-15: Memory, Instruction Format, Addressing Modes, Registers

Your instructor covered: 4096x16 memory, 3-bit opcode + 12-bit address + I bit format, direct and indirect addressing, and a list of all 8 registers with their bit widths and purposes.

What your report needs to show for DUAD-16:

- 512x16 memory table with address width justification (9 bits because 2^9 = 512)
- The 16-bit instruction format diagram showing all 4 fields with bit positions labeled
- All 4 addressing modes with examples, using the same diagram style your instructor used in Lecture 14 (the direct vs indirect addressing diagram with AC and the memory arrows)
- A register table: R0-R7 (16 bits each), PC (9 bits), IR (16 bits), with purpose of each

2.2 From Lecture 16: Instruction Set Completeness and Common Bus

Your instructor defined that a complete instruction set must cover: Arithmetic/Logic/Shift, Data Transfer, Program Control, and I/O. He also showed how the common bus is built using multiplexers.

What your report needs:

- A section explicitly stating that DUAD-16's instruction set is complete, listing which instructions cover each of the four required categories
- A common bus diagram for DUAD-16 showing what connects to the bus and how many MUX lines are needed
- The ALU table (like the one in Lecture 16) showing S3 S2 S1 S0 control signals and what operation each combination produces

2.3 From Lecture 17-18: Instruction Cycle and Control Unit

Your instructor showed the full instruction cycle (Fetch, Decode, Indirect, Execute) as a flowchart, and the Control Unit block diagram with a 3x8 decoder + 4x16 decoder + sequence counter SC.

What your report needs:

- A flowchart of the DUAD-16 instruction cycle using the same style (Start -> T0 -> T1 -> T2 -> Decode -> Type? -> Execute)
- The Control Unit block diagram for DUAD-16 showing: IR feeding into a 5-to-32 decoder (for 5-bit opcode), MODE bits going to addressing logic, DST/SRC bits going to the register file selector, and the sequence counter
- Explanation of how your decoder works differently from the Mano Machine's 3x8 decoder

2.4 From Lectures 19-21: Memory Reference, Register Reference, and I/O Instructions

Your instructor wrote out every single instruction as microoperations with timing signals. This is the most detailed part of the Mano Machine description and your instructor clearly cares about it.

What your report needs:

- Microoperation sequences for ALL 20 DUAD-16 instructions written in the same T0, T1, T2... notation
- For register-reference type instructions (like NOT, INC, DEC), explain how they are identified and executed
- For memory-reference instructions (LDM, STM), show the full sequence including the memory read/write
- For I/O instructions, describe the flag-based program-controlled I/O method

2.5 From Lecture 22: Interrupts and Complete Computer Description

Your instructor ended with the complete computer description including the interrupt cycle (IEN, FGI, FGO flags, RT0, RT1, RT2 microoperations) and a full flowchart.

What your report needs:

- A description of DUAD-16's interrupt mechanism (adapted from the Mano Machine interrupt cycle)
- A complete computer description section that summarizes all microoperations in one place, like the table in Lecture 22
- The complete flowchart for DUAD-16 from Start to Execute, matching the style of the complete computer flowchart shown in Lecture 22

PART 3: 10-Day Workflow — Day by Day

You have from today until April 20. That is 13 days. Here is a day-by-day plan that gets everything done with time to spare for printing and Turnitin.

Day	Date	Usman (CPU Architecture + Report)	Reyan (Simulator Frontend)	Saim (Simulator Engine + Manual)	Group Task
Day 1	Apr 8 (Today)	Re-read Lectures 14-22 fully. Understand every diagram your instructor drew. Note which diagrams you need to recreate for DUAD-16.	Download the simulator HTML file already built. Open it in browser, test the sample program. Understand what each panel does.	Read the JavaScript code in the simulator. Understand how the assembler works. List any bugs or missing instructions.	Meet for 1 hour. Agree on the final instruction set (use the 20 instructions already designed). Divide sections clearly.
Day 2	Apr 9	Draw the DUAD-16 datapath block diagram on paper first, then recreate in PowerPoint or draw.io. This is the main architecture diagram.	Improve the simulator CSS: make fonts bigger, add a title with your institute name, add your names in the header.	Test all 20 instructions in the simulator one by one. Write down which ones work correctly and which need fixing.	Share progress on WhatsApp group. No meeting needed.
Day 3	Apr 10	Write Section 1 of the report: Introduction and motivation. Write Section 2: Memory and Register Organization (with tables).	Add a 'Help / Instruction Reference' panel to the simulator showing all 20 instructions with their opcodes.	Fix any failing instructions in the JS engine. Confirm ADD, SUB, MUL, LDM, STM, JZ, JNZ all work correctly with sample programs.	All three test the simulator together for 30 minutes.
Day 4	Apr 11	Write Section 3: Instruction Format (with the 16-bit format diagram). Write Section 4: Addressing Modes with examples for all 4 modes.	Add the instruction manual panel. Polish the layout. Make sure it works on different screen sizes.	Write all microoperation sequences for 20 instructions on paper (T0, T1, T2 notation). This is needed by Usman for the report.	Saim sends Usman the microoperation table by end of day.
Day 5	Apr 12	Write Section 5: Instruction Set	Begin writing the Simulator Manual document separately	Continue writing microoperations.	Mid-project check: Is the

Day	Date	Usman (CPU Architecture + Report)	Reyan (Simulator Frontend)	Saim (Simulator Engine + Manual)	Group Task
		(complete table of all 20 instructions). Write Section 6: ALU Design (with operation table and circuit description).	as a Word file: what each button does, how to load a program, what each panel shows.	Add INP/OUT instruction support to the simulator (program-controlled I/O using R0 convention).	simulator fully working? Is the report at least 1500 words? If behind, discuss.
Day 6	Apr 13	Write Section 7: Control Unit Design (decoder block diagram, sequence counter description, how instruction type is determined).	Finish the Simulator Manual first draft. Hand it to Saim to review.	Review Simulator Manual. Write the Interrupt section description for the report and send to Usman.	Rest / catch up day if needed.
Day 7	Apr 14	Write Section 8: Instruction Cycle and Microoperations (full fetch-decode-execute flowchart + all T notation microops from Saim's table).	Final polish of the simulator. Test on Chrome, Firefox, Edge. Fix any last visual issues.	Write the Abstract for the report (150-200 words summarizing the novelty of DUAD-16). Write the Bibliography.	Usman completes the full first draft of the report today.
Day 8	Apr 15	Write Section 9: I/O and Interrupts. Write Section 10: Complete Computer Description (the summary table of all microoperations). Proofread entire report.	Add one more sample program to the simulator (e.g., finding maximum of two numbers). This makes the demo more impressive.	Proofread the full report. Check for grammar. Verify all microoperations are correct. Check word count is above 3000.	Full team review meeting: read the report together, fix anything unclear.
Day 9	Apr 16	Format the report: front cover, consistent heading styles, page numbers, bibliography. Save as PDF to check layout.	Final simulator testing. Export the HTML file as the final version DUAD16_Simulator_Final.html	Write the simulator manual final version. Make sure it includes: how to run, all 20 instructions with syntax, two example programs.	Take the printed draft to the Librarian for Turnitin check. Get it stamped.
Day 10	Apr 17	Fix anything the Turnitin report	Print the simulator manual.	Prepare the presentation	Final team rehearsal of

Day	Date	Usman (CPU Architecture + Report)	Reyan (Simulator Frontend)	Saim (Simulator Engine + Manual)	Group Task
		flags. Rewrite any section above 19% similarity in your own words.		slides: 8-10 slides covering the design, the simulator demo, and comparison with Mano Machine.	the presentation. Each person practices their section.
Days 11-13	Apr 18-20	Final printing and soft binding of the report. Make sure front cover has all required info. Submit before April 20 deadline.	Bring a laptop to the presentation with the simulator ready to demo.	Bring the printed manual to submit with the report.	SUBMIT the report to the instructor's office before 5pm on April 20.

PART 4: What the Final Report Must Contain (Section by Section)

The rubric says your report must follow the structure of Chapter 5 of the Mano textbook. Here is that structure mapped to DUAD-16, with the minimum content for each section.

Front Cover (required, do not forget anything)

- Project title: DUAD-16: A 2-Address 16-bit Processor with General-Purpose Register File
- Institute name and logo (Ghulam Ishaq Khan Institute of Engineering Sciences and Technology)
- Faculty name: Prof. Dr. G. Abbas
- Course: CE222 – Computer Organization and Assembly Language
- Section: your section number
- Group members: Full names and registration numbers of Usman, Reyan, Saim
- Date: April 2026

Abstract (150-200 words, written by Saim)

Write this in your own words. It must answer three questions: What did you design? How is it different from the Mano Machine? What does the simulator demonstrate? Do NOT use AI language like 'comprehensive', 'robust', 'leveraging'. Write simply and directly.

Theory and Design (minimum 3000 words, main body, written by Usman)

This must contain all of these sub-sections:

- 1. Introduction: Why we designed DUAD-16 and what problem it solves compared to the Mano Machine
- 2. Memory Organization: 512x16 RAM, 9-bit address, program zone vs data zone
- 3. Register Organization: Table of all registers (R0-R7, PC, IR) with bit widths and purpose
- 4. Instruction Format: The 16-bit format diagram with all 4 fields labeled, worked example of ADD R2, R5 encoded in binary
- 5. Addressing Modes: All 4 modes with diagrams showing how effective address is calculated
- 6. Instruction Set: Complete table of all 20 instructions with opcode, operation, and category. Prove completeness against the 4 required categories.
- 7. ALU Design: The operation table (S3 S2 S1 S0 Cin -> Function), the circuit block diagram, how two register values are fed in and result written back
- 8. Control Unit Design: The 5-to-32 decoder, sequence counter, how instruction type is determined, the complete CU block diagram
- 9. Common Bus System: What connects to the bus, how the MUX structure works, the bus diagram
- 10. Instruction Cycle: The flowchart (Fetch -> Decode -> Execute), all timing signals, complete microoperation table for all 20 instructions
- 11. I/O and Interrupts: Program-controlled I/O description, interrupt mechanism (IEN flag, interrupt cycle)
- 12. Complete Computer Description: The summary flowchart and the full microoperation list

Bibliography

- Morris Mano, Computer System Architecture, 3rd Edition, Prentice Hall
- Patterson and Hennessy, Computer Organization and Design, 5th Edition (for RISC reference)
- Any websites you used (cite with full URL and access date)

PART 5: Microoperation Reference for the Report

This is the most technical part of the report. Your instructor writes microoperations for every instruction. Below are the DUAD-16 microoperations in your instructor's notation. Usman must include all of these in Section 10 of the report.

5.1 Fetch and Decode (same for all instructions)

```
T0:  AR <- PC
T1:  IR <- M[AR],  PC <- PC + 1
T2:  OPCODE <- IR[15:11],  MODE <- IR[10:9],  DST <- IR[8:6],  SRC <- IR[5:0]
     Decode OPCODE using 5-to-32 decoder
```

5.2 Register-to-Register Instructions (MODE = 00)

For ADD, SUB, MUL, AND, OR, XOR, MOV the execution is:

```
T3:  A <- RF[DST],  B <- RF[SRC]     (read both registers from
register file)
T4:  RF[DST] <- ALU_result,  set Z/C/N flags,  SC <- 0
```

The specific ALU operation at T4 depends on the opcode. For ADD: result = A + B. For AND: result = A AND B. And so on.

5.3 Load Immediate (LDI Rd, #imm)

```
T3:  RF[DST] <- IR[5:0]  (the 6-bit immediate field),  SC <- 0
```

5.4 Load from Memory (LDM Rd, [Rs])

```
T3:  AR <- RF[SRC]          (address is in register Rs)
T4:  DR <- M[AR]            (read from memory)
T5:  RF[DST] <- DR,  SC <- 0
```

5.5 Store to Memory (STM [Rd], Rs)

```
T3:  AR <- RF[DST]          (address is in register Rd)
T4:  M[AR] <- RF[SRC],  SC <- 0
```

5.6 Jump Unconditional (JMP addr)

```
T3:  PC <- IR[8:0],  SC <- 0    (9-bit address from instruction)
```

5.7 Jump if Zero (JZ Rd, addr)

```
T3:  if (RF[DST] = 0) then PC <- IR[5:0]
     SC <- 0
```

5.8 Jump if Not Zero (JNZ Rd, addr)

```
T3:  if (RF[DST] != 0) then PC <- IR[5:0]
     SC <- 0
```

5.9 Single-Register Instructions (NOT, INC, DEC)

```
T3:  A <- RF[DST]
T4:  RF[DST] <- ALU_result (NOT A / A+1 / A-1),  SC <- 0
```

5.10 Shift Instructions (SHL Rd, #n and SHR Rd, #n)

```
T3:  A <- RF[DST],  n <- IR[3:0]   (shift amount from lower 4 bits of
SRC field)
```

```
T4:  RF[DST] <- A shifted by n,  set C flag if bits shifted out,  SC
<- 0
```

5.11 Compare (CMP Rd, Rs)

```
T3:  A <- RF[DST],  B <- RF[SRC]
T4:  temp <- A - B,  set Z/C/N flags only (do NOT write result back),
SC <- 0
```

5.12 Halt (HLT)

```
T3:  S <- 0,  SC <- 0   (S is the start-stop flip-flop, same as Mano
Machine)
```

5.13 Interrupt Cycle (adapted from Mano Machine Lecture 22)

Condition to trigger interrupt: T0.T1.T2 . IEN . (FGI + FGO): R <- 1

```
RT0:  AR <- 0,  TR <- PC
RT1:  M[AR] <- TR,  PC <- 0
RT2:  PC <- PC + 1,  IEN <- 0,  R <- 0,  SC <- 0
```

This is identical in structure to the Mano Machine interrupt cycle from Lecture 22. The difference is that DUAD-16 uses a register-based I/O convention: INP loads data into R0, OUT outputs from R0.

PART 6: Quick Reference Checklist Before Submission

Before you print the report, go through this checklist:

Report checklist

- Front cover complete with all required fields
- Turnitin report attached, similarity under 19%, AI index acceptable, librarian signature and stamp present
- Abstract written in plain student language, mentions novelty clearly
- Theory section is minimum 3000 words (use Word's word count tool)
- Instruction format diagram is present and labeled with bit positions
- ALU design table and block diagram are present
- Control Unit block diagram is present
- Common bus diagram is present
- All 20 instructions are in the instruction set table
- All 4 addressing modes are explained with examples
- Microoperation sequences are written for all instructions
- Fetch-decode-execute flowchart is present
- Complete computer description section is present

- I/O and interrupt section is present
- Simulator manual is attached
- Bibliography has at least 3 sources properly cited
- Soft binding done, A4 paper

Simulator checklist

- Opens in browser without any errors
- All 20 instructions assemble correctly
- Step and Run buttons work
- Register values update visually after each instruction
- Flags (Z, C, N) update correctly
- Sample program runs and gives correct result in R3 = 15
- Speed slider works
- Your names and institute in the header

Presentation checklist

- All 3 members have slides to present
- Usman presents: CPU architecture, instruction format, ALU/CU design
- Reyan presents: Simulator demo (live in browser), bus diagram, memory organization
- Saim presents: Instruction set, microoperations, I/O and interrupt handling
- Each member can answer questions about their own section
- Simulator is open and ready on a laptop during the presentation

---

Good luck Usman, Reyan, and Saim. You have everything you need.